

Kod, który słyhać – ściąga z rozwiązań

Co zmienia się między szkieletem kroku (step-N) a rozwiązaniem (step-N-final) w src/main. Nad każdą zmianą sygnatura metody, w której leży; liczba z lewej to numer linii w step-N-final. Ścieżki plików od pl/livecoding/musicjam/.

+ rozwiązanie

- [n linii TODO] ze szkieletu

1. [Krok 2: Czytamy plik MIDI](#)
2. [Krok 3: Własny scheduler i Gervill](#)
3. [Krok 4: Zewnętrzny syntezytor](#)
4. [Krok 5: Perkusja, dwa zegary i jeden zegar](#)

Stan gałęzi: step-5-final @ ab3c5e5 2026-09-16

Krok 2: Czytamy plik MIDI

Uruchom: step2.InspectMidi

Testy: MidiFileReaderTest (9, zielone od startu 4)

```
midi/MidiFileReader.java + import pl.livecoding.musicjam.model.Voice,  
java.util.Comparator
```

```
58     private static TrackReading readTrack(Track track, int index, int resolution) {  
        :  
        } else if (message instanceof ShortMessage shortMessage) {  
- [4 linie TODO]  
+         int pitch = shortMessage.getData1();  
+         if (channel.isEmpty()) {  
+             channel = OptionalInt.of(shortMessage.getChannel());  
+         }  
+         if (shortMessage.getCommand() == ShortMessage.PROGRAM_CHANGE &&  
!programSeen) {  
+             program = shortMessage.getData1();  
+             programSeen = true;  
+         } else if (isNoteOn(shortMessage)) {  
+             playingNotes.start(pitch, event.getTick(),  
shortMessage.getData2());  
+         } else if (isNoteOff(shortMessage)) {  
+             playingNotes.finish(pitch)  
+                 .map(start -> noteBetween(start, pitch, event.getTick(),  
resolution))  
+                 .ifPresent(notes::add);  
+         }  
    }
```

```
58     private static TrackReading readTrack(Track track, int index, int resolution) {  
        :  
- [2 linie TODO]  
+     notes.sort(Comparator.comparingDouble(Note::beat)  
+         .thenComparingInt(note -> ((Voice.Pitch) note.voice()).midiNote()));  
        return new TrackReading(new TrackData(index, name, channel, program,  
List.copyOf(notes)), bpm);
```

```
100 private static Note noteBetween(  
    PlayingNotes.PlayingNote start, int pitch, long endTick, int resolution)  
    {  
- [4 Linie TODO]  
+     double durationBeats = ticksToBeats(endTick - start.tick(), resolution);  
+     if (durationBeats <= 0.0) {  
+         return null;  
+     }  
+     return new Note(  
+         ticksToBeats(start.tick(), resolution),  
+         new Voice.Pitch(pitch),  
+         durationBeats,  
+         start.velocity() / 127f);  
    }
```

Krok 3: Własny scheduler i Gervill

Uruchom: step3.ListMidiDevices, step3.PlayNotes

Testy: MidiNoteOutputTest, MidiPlayerTest, PooledSchedulerTest

midi/MidiNoteOutput.java

```
29     static MidiNoteOutput open(Synthesizer synthesizer, int channelNumber, int
        program)
            throws MidiUnavailableException {
- [4 linie TODO]
+     synthesizer.open();
+     MidiChannel channel = channelOf(synthesizer, channelNumber);
+     channel.programChange(program);
+     return new MidiNoteOutput(synthesizer, channel);
    }

66     public synchronized void noteOn(int pitch, int velocity) {
- [2 linie TODO]
+     channel.noteOn(pitch, velocity);
    }

71     public synchronized void noteOff(int pitch) {
- [2 linie TODO]
+     channel.noteOff(pitch);
    }
```

```
midi/MidiPlayer.java + import pl.livecoding.musicjam.model.Voice,  
java.util.ArrayList
```

```
64     static List<ScheduledEvent> schedule(  
        :  
        }  
- [5 lines TODO]  
+     var events = new ArrayList<ScheduledEvent>(Math.multiplyExact(notes.size(),  
        2 * loops));  
+     for (int loop = 0; loop < loops; loop++) {  
+         double loopStart = loop * patternLengthBeats;  
+         for (Note note : notes) {  
+             int pitch = ((Voice.Pitch) note.voice()).midiNote();  
+             int velocity = Math.round(note.velocity() * 127.0f);  
+             events.add(ScheduledEvent.noteOn(  
+                 beatToNanos(loopStart + note.beat(), bpm), pitch,  
+                 velocity));  
+             events.add(ScheduledEvent.noteOff(  
+                 beatToNanos(loopStart + note.beat() + note.durationBeats(),  
+                 bpm), pitch));  
+         }  
+     }  
+     return events;  
}
```

```
scheduler/PooledScheduler.java
```

```
29     public void submit(long offsetNanos, TimedTask task) {  
- [5 lines TODO]  
+         long targetNanos = startNanos + offsetNanos;  
+         pool.schedule(  
+             () -> task.runAt(targetNanos),  
+             targetNanos - System.nanoTime(),  
+             TimeUnit.NANOSECONDS);  
}
```

Krok 4: Zewnętrzny syntezytor

Uruchom: step4.PlayOnSynth (awaryjnie step4.MidiPanic)

Testy: ExternalMidiOutputTest (9, zielone od startu 3)

midi/ExternalMidiOutput.java + import java.util.Locale

```
88     static MidiDevice find(String nameContains, List<MidiDevice> devices) throws
        MidiUnavailableException {
- [5 linii TODO]
+     String wanted = nameContains.toLowerCase(Locale.ROOT);
+     for (MidiDevice device : devices) {
+         String name = device.getDeviceInfo().getName().toLowerCase(Locale.ROOT);
+         if (name.contains(wanted) && device.getMaxReceivers() != 0) {
+             return device;
+         }
+     }
+     throw new MidiUnavailableException("No MIDI device named like \"" +
        nameContains + "\" takes messages");
    }

100    public synchronized void noteOn(int pitch, int velocity) {
- [2 linie TODO]
+     send(ShortMessage.NOTE_ON, pitch, velocity);
    }

105    public synchronized void noteOff(int pitch) {
- [2 linie TODO]
+     send(ShortMessage.NOTE_OFF, pitch, 0);
    }

109    private void send(int command, int data1, int data2) {
- [4 linie TODO]
+     try {
+         receiver.send(new ShortMessage(command, channel, data1, data2), -1);
+     } catch (InvalidMidiDataException exception) {
+         throw new IllegalArgumentException(exception);
+     }
    }
```

```
123 private synchronized void allSoundOff() {  
- [3 Linie TODO]  
+     try {  
+         send(ShortMessage.CONTROL_CHANGE, ALL_SOUND_OFF, 0);  
+     } catch (RuntimeException deviceAlreadyGone) {  
+         // best effort: by the time a shutdown hook runs, the device may be  
+         closed or unplugged  
+     }  
+ }
```

Krok 5: Perkusja, dwa zegary i jeden zegar

Uruchom: step5.PlayBeat, step5.PlayJam

Testy: TransportTest, PatternCompilerTest, AudioEngineScheduleTest, AudioEngineTest, MidiPlayerLoopTest

audio/AudioEngine.java + import java.util.ArrayList

```
225     static List<Hit> schedule(List<Note> notes, double bpm, double
        patternLengthBeats, int loops, int sampleRate) {
        :
        }
- [5 linii TODO]
+     Transport transport = new Transport(bpm, sampleRate);
+     var hits = new ArrayList<Hit>(notes.size() * loops);
+     for (int loop = 0; loop < loops; loop++) {
+         double loopStart = loop * patternLengthBeats;
+         for (Note note : notes) {
+             long frame = transport.frameAtBeat(loopStart + note.beat());
+             hits.add(new Hit(frame, note));
+         }
+     }
+     return hits;
    }
```

audio/Transport.java

```
18     public long frameAtBeat(double beat) {
- [2 linie TODO]
+     return Math.round(beat * 60.0 / bpm * sampleRate);
    }
```

midi/MidiPlayer.java

```
78     public void playLoop(List<Note> notes, double bpm, double patternLengthBeats,
        int loops,
        :
- [5 linii TODO]
+     for (int loop = 0; loop < loops; loop++) {
+         long startNanos = loopStartNanos(
+             System.nanoTime(), heardNanos.getAsLong(), loop, loopNanos,
+             latencyNanos);
+         playOneLoop(oneLoop, startNanos);
+     }
    }
```

```
model/PatternCompiler.java + import java.util.ArrayList,  
java.util.Comparator
```

```
18 public static List<Note> compile(List<DrumTrack> tracks, int beatsPerBar) {  
- [4 Linie TODO]  
+     var notes = new ArrayList<Note>();  
+     for (DrumTrack track : tracks) {  
+         String steps = track.steps();  
+         double stepBeats = (double) beatsPerBar / steps.length();  
+         for (int step = 0; step < steps.length(); step++) {  
+             double beat = step * stepBeats;  
+             parse(steps.charAt(step)).ifPresent(accent ->  
+                 notes.add(new Note(beat, track.drum(), stepBeats,  
+ track.gain() * accent)));  
+         }  
+     }  
+     notes.sort(Comparator.comparingDouble(Note::beat));  
+     return List.copyOf(notes);  
}
```